Vision Transformer Implementation

DATA SETUP:

- Normalized Data with
- cifar10_mean = [0.4914, 0.4822, 0.4465]
- cifar10_std = [0.2023, 0.1994, 0.2010]
- Random Cropping, Random Horizontal Flip and Random Augmentation Applied to data
- Batch Size 128

MODEL LAYERS :

1. Patch Embedding :

- Patch Embedding , creates patches and returns Tensor of Dimension: 128 X 197 X 128 : 128 images 196 tokens (patches + CLS) per image, 128 dimensions per token (128-dimension vector per patch)
- Using Convolution 2d as Projection Layer as it's more efficient than a Linear Layer. The paper actually Mentions a Linear Projection method in main approach and Convolution Projection as an alternative approach. kernel and stride are assigned to patch size as we want patches and not overlaps. For square images number of patches is**: (image size/patch size) ^2** (CIFAR10 is square images)
- The CLS token (classification token) in Vision Transformers is a special learnable parameter that serves as a placeholder for aggregating global information from all image patches for classification tasks
- Positional Embedding is added for the image patches in an element-wise addition operation
- Tensor Output from Patch Embedding Operation: [128, 197, 384] : 128 images 197 tokens per image (CLS token + 196 Patches), 128 dimension representation of each token
- Kernel Size = Stride = Patch Size = 4
- Patch Embedding Block:
- PatchEmbedding((projection): Conv2d(3, 384, kernel_size=(4, 4), stride=(4, 4)))

2. Transfomer Encoder:

- Each head processes Embedding Size/ Number of Heads Dimensions
- Pre Norm Architecture (Norm before adding to Multi Head Attention as per VIT Paper) stabilizes gradients during training, enables training of deeper networks also
- Normalizing before each major operation to prevent gradient Explosion and Vanishing Gradients during Back Prop
- Q K V for Multihead Self Attention come from the same source i.e the normalized tensor
- Residual Connection: Adds Original Input to the output from Multi Head Attention
- Output is passed through the MLP described below
- 12 Transformer Blocks Used

Transformer Block:

```
TransformerEncoder(
  (norm1): LayerNorm((384,), eps=1e-05, elementwise_affine=True)
  (multiHeadAttn): MultiheadAttention(
   (out_proj): NonDynamicallyQuantizableLinear(in_features=384,
out_features=384, bias=True)
  )
  (norm2): LayerNorm((384,), eps=1e-05, elementwise_affine=True)
  (mlp): MLP(
   (fc1): Linear(in_features=384, out_features=512, bias=True)
   (fc2): Linear(in_features=512, out_features=384, bias=True)
   (dropout): Dropout(p=0.1, inplace=False)
  )
```

3. MLP Layer :

- Position-wise feed-forward network MLP: Applies Non Linear Transformation on the token generated from Multi Head Attention
- MLP applies the same transformation independently to each of the 197 tokens (196 Patches + CLS Token)
- Performs an EXPAND ACTIVATE PROJECT BACK operation, Maintains original Dimensions
- Gaussian Error Linear Unit Activation Function with DROPOUT_RATE=0.1

4. Linear Classification Head:

- Linear Classification Head performs logits=W·x+b , Output is logits of num classes size vector (10 for Cifar10)
- CLS Token is Extracted and passed to the Linear Classification Head.

Classification Head:

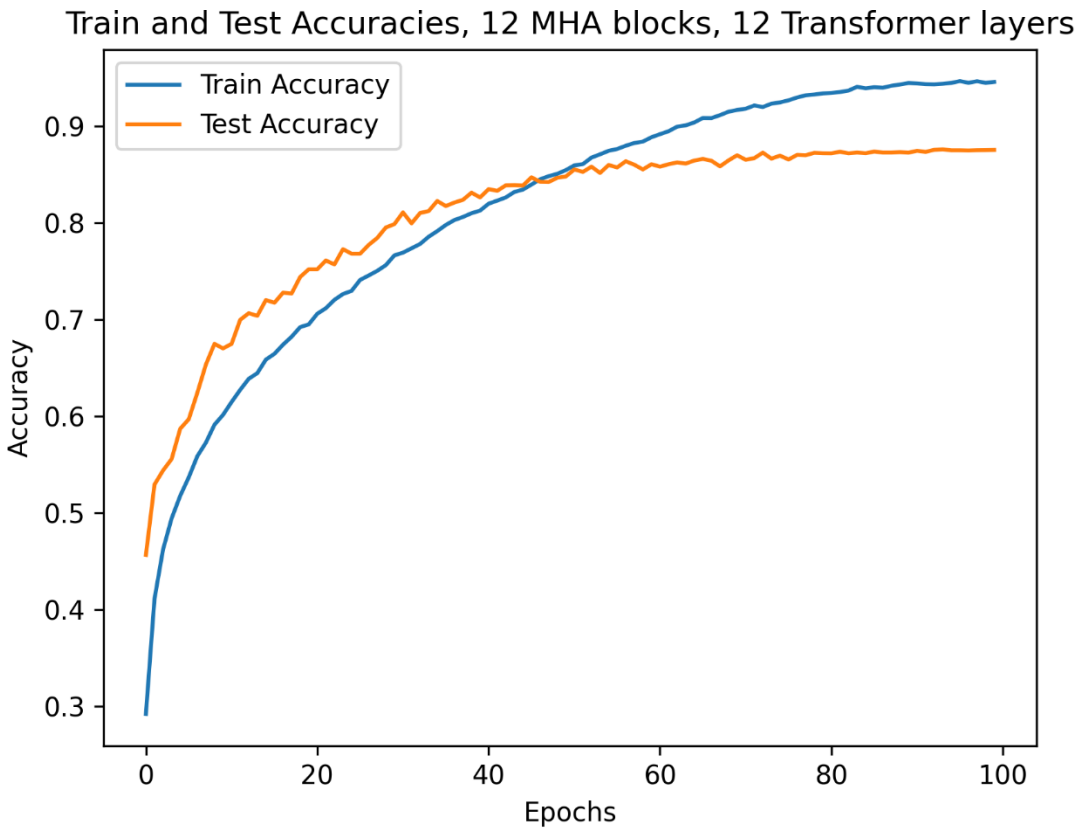
Linear(in_features=384, out_features=10, bias=True)

Training Setup:

- Loss Function is Cross Entropy Loss
- AdamW Optimizer with LEARNING_RATE = 3e-4, betas= (0.9, 0.999), weight_decay=0.01
- Cosine Annealing Learning Rate Scheduler

Results:

After 100 Epochs : Train Loss : 0.1559 , Train Accuracy : 0.9456, Test Accuracy : 0.8753



- Test Accuracy: 0.88%
- Precision: 0.88
- Recall: 0.88
- F1 Score: 0.88